第四届eBPF开发者大会

The 4th eBPF Developer Conference (eBPFDC 2026)

陕西 · 西安　　2026年4月18日

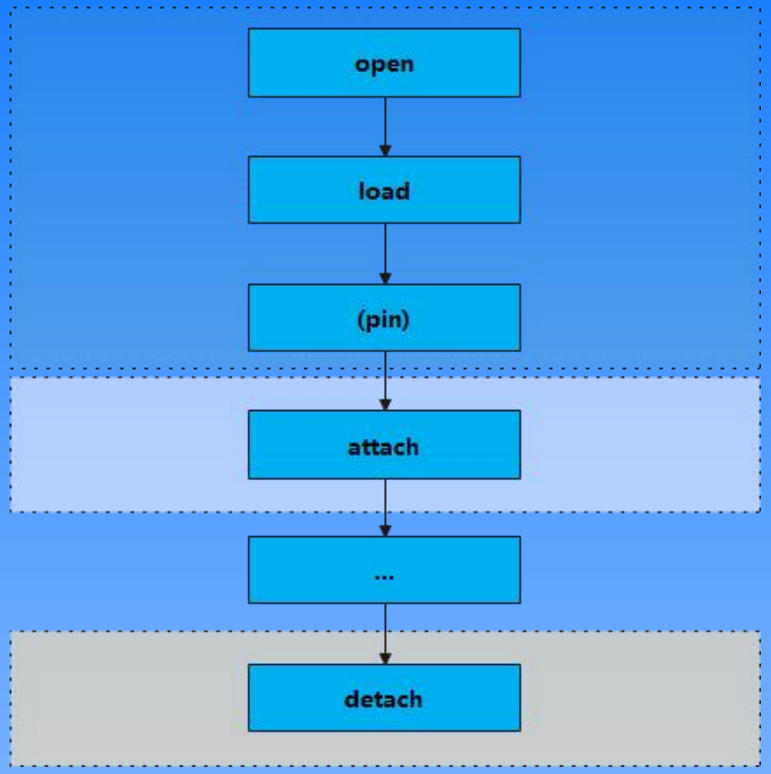
https://www.ebpftravel.com

sched_ext 商用部署实战

刘文方 梁东旭 王子成 刘璐

陕西 · 西安 2026年4月18日

普通 eBPF 程序的加载方式和安卓规则下基于sched_ext的eBPF scheduler加载差异：

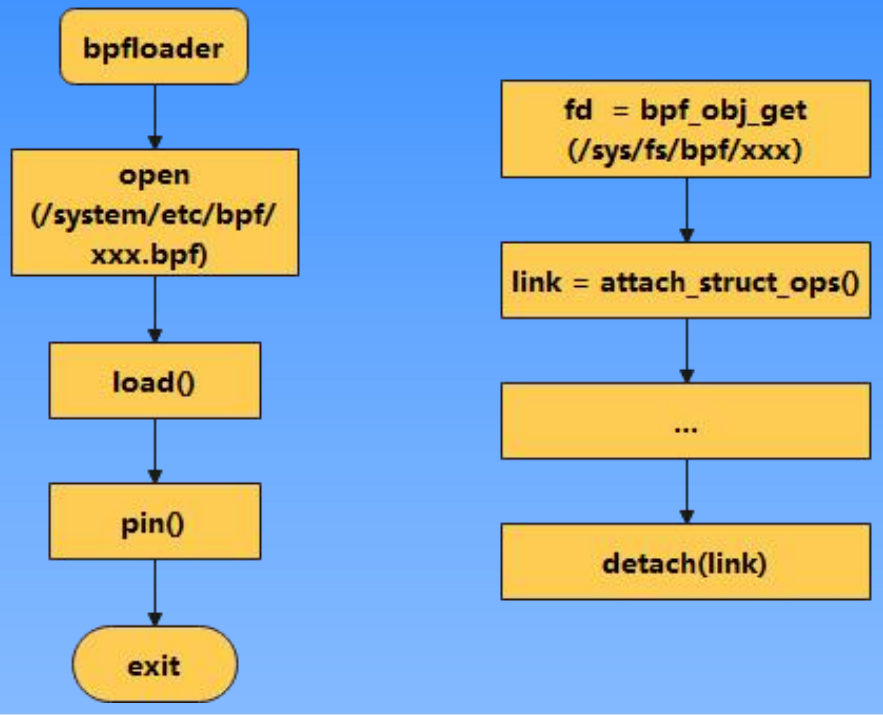


1，安卓bpfloader对sched_ext依赖的STRUCT_OPS未做支持，补全了对应加载链路。

2，安卓新版本内核STRUCT_OPS的生命周期被 bpf_link 接管，必须在加载的时候对map设置BPF_F_LINK flag，避免attach时bpf_link_create失败。
bpf_map__set_map_flags(map, BPF_F_LINK);

sched_ext 在终端的应用场景扩展出各种用户态调度器，其中以游戏调度器为例，其生命周期：

- 手机启动阶段资源文件被bpfloader一次加载完成，通过 bpf_obj_pin将eBPF 对象（prog 和 map）固定到指定文件系统（比如/sys/fs/bpf）；
- 此后仅在用户切换到对应游戏时，同步切换到游戏调度器：获取到eBPF 对象（获取map fd和 prog fd等信息）后，执行attach操作，完成包含调度操作的 STRUCT_OPS 链接到 sched_ext 框架；
- 用户退出游戏时同步退出游戏调度器：执行bpf_link__destroy(link)操作。



Android eBPF 生态同时存在两套宏体系与构建体系：Android 自身的 BPF 宏框架和通用 libbpf 宏框架；在Android Soong 构建系统（Android.bp）中，其分别对应不同的模块类型：

- bpf {}：使用 Android 定制的 BPF 宏体系
- libbpf_prog {}：使用 libbpf 宏体系

两种都会依赖：

- system/bpf/include/defs/android_bpf_defs.h

但在编译阶段会根据ENABLE_LIBBPF 宏而走不同的依赖路径：

- bpf {} 默认使用 Android 自定义的 BPF helper 与宏体系。
- libbpf_prog {} 默认切换到 libbpf 编程模型，并使用 vmlinux BTF 与标准的 SEC() 等宏。此时对 bpf_helpers.h 的依赖需要被导向：external/libbpf/src/bpf_helpers.h

item	bpf	libbpf_prog
Soong module	bpf	libbpf_prog
编译宏	无	ENABLE_LIBBPF
helper	Android helper	libbpf
map/prog 宏	DEFINE_BPF_*	SEC(".maps")
头文件	system/bpf	libbpf headers

在当前的实现中，选择使用 libbpf_prog{} 构建方式。该方式通过启用 libbpf + vmlinux BTF，使 eBPF 程序能够使用通用 libbpf 接口与结构定义，从而满足 sched_ext 所需的 STRUCT_OPS等特性，并完成 eBPF scheduler 的编译构建。

使用libbpf 宏框架开发：
比如在system/bpfprogs/目录中扩展simple.c eBPF scheduler程序文件，并在对应的 Android.bp 中添加 libbpf_prog {} 构建配置：

system/bpfprogs/Android.bp增加：

```
libbpf_prog {
    name: "simple.bpf",
    srcs: ["simple.c"],
    header_libs: [
        "bpf_prog_headers",
        "libcutils_headers",
        "android_bpf_defs",
    ],
}
```

simple.c 实现：

```
void BPF_STRUCT_OPS(simple_stopping, struct task_struct *p,
bool runnable)
…
SEC(".struct_ops")
struct sched_ext_ops simple_vip_ops = {
        .select_cpu          = (void *)simple_select_cpu,
        .enqueue             = (void *)simple_enqueue,
        .dispatch            = (void *)simple_dispatch,
        .running             = (void *)simple_running,
        .stopping            = (void *)simple_stopping,
        .enable              = (void *)simple_enable,
        .init                = (void *)simple_init,
        .exit                = (void *)simple_exit,
        .name                = "simple_vip",
};
```

编译生成simple.bpf，会被放到终端系统指定位置system/etc/bpf/，下次重启过程由bpfloader加载。

Android平台下基于bcc编译skeleton架构模式bpf程序在开发调测过程中具有很多便利：
新增eBPF程序simple.bpf.c 用以实现调度器框架；
新增用户态程序simple.c，其只需 #include “simple.skel.h"，然后调用 skeleton API 就能加载并运行内核中的 BPF 程序。

Android.bp中改动：

```
cc_object {
    name: "simple.bpf.o",
    srcs: ["simple.bpf.c"],
    defaults: ["bpf_kernel_defaults", "vmlinux_header"],
}

genrule {
    name: "simple.skel.h",
    srcs: [":simple.bpf.o"],
    out: [" simple.skel.h"],
    defaults: ["bpf_skeleton_hdr_defaults"],
}

cc_binary {
    name: " simple",
    srcs: ["simple.c"],
    generated_headers:  ["simple.skel.h"],
    defaults: ["bpf_binary_defaults"],
}
```

simple.bpf.c 实现：

```
void BPF_STRUCT_OPS(simple_stopping, struct task_struct *p,
bool runnable)
…
SEC(".struct_ops")
struct sched_ext_ops simple_vip_ops = {
        .select_cpu          = (void *)simple_select_cpu,
        .enqueue             = (void *)simple_enqueue,
        .dispatch            = (void *)simple_dispatch,
        .running             = (void *)simple_running,
        .stopping            = (void *)simple_stopping,
        .enable              = (void *)simple_enable,
        .init                = (void *)simple_init,
        .exit                = (void *)simple_exit,
        .name                = "simple",
};
```

编译过程：

simple.bpf.c
▼
simple.bpf.o (BPF 目标)
▼
simple.skel.h (bpftool gen skeleton)
▼
simple.c (用户代码)
trace_helpers.c (eBPF loader关联文件)
btf_helpers.c
uprobe_helpers.c
android/argp.cpp
▼
多个 .o（native arm64）
▼
链接（ld.lld）
▼
unstripped/simple
▼
strip
▼
simple

.bpf.o规则，Soong 使用 Android clang + BPF backend：

```
clang -target bpf -D__TARGET_ARCH_arm64 \
-I out/soong/.intermediates/external/libbpf/libbpf_headers/gen \
-fno-stack-protector -fno-unwind-tables \
-mcpu=generic -mllvm -bpf-stack-size=1024 \
-g \
-c simple.bpf.c  -o simple.bpf.o
```

.skel.h 规则：
借助Host 工具 bpftool，生成simple.skel.h

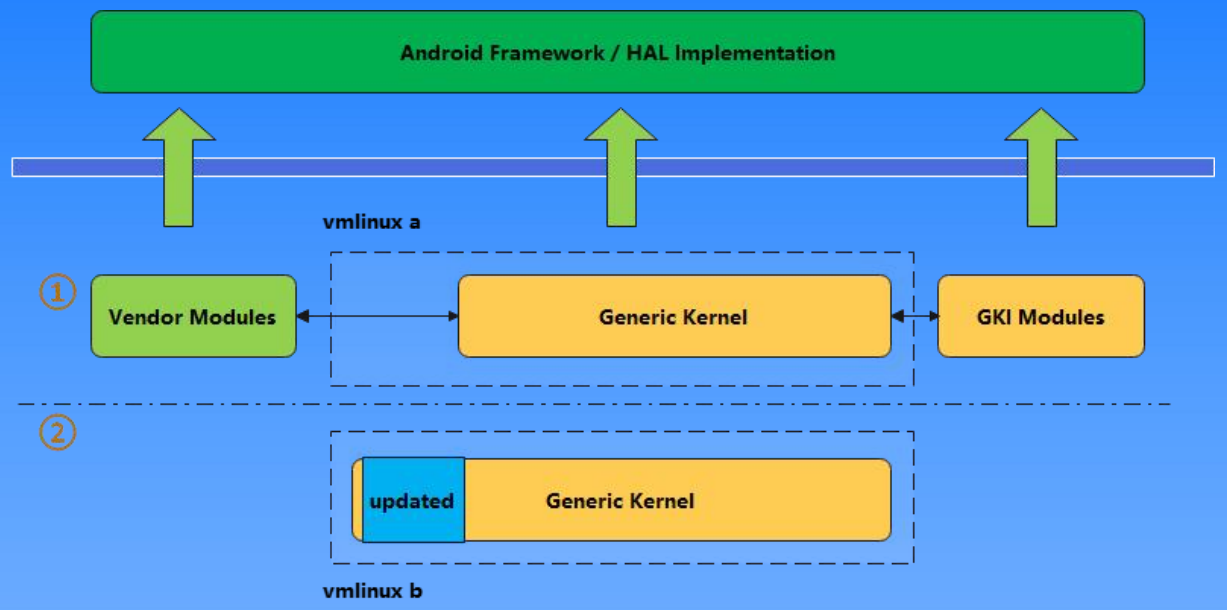
```
genrule_defaults {
    name: "bpf_skeleton_hdr_defaults",
    tools: [“bpftool”],
    cmd: "$(location bpftool) gen skeleton $(in) > $(out)",
}
```

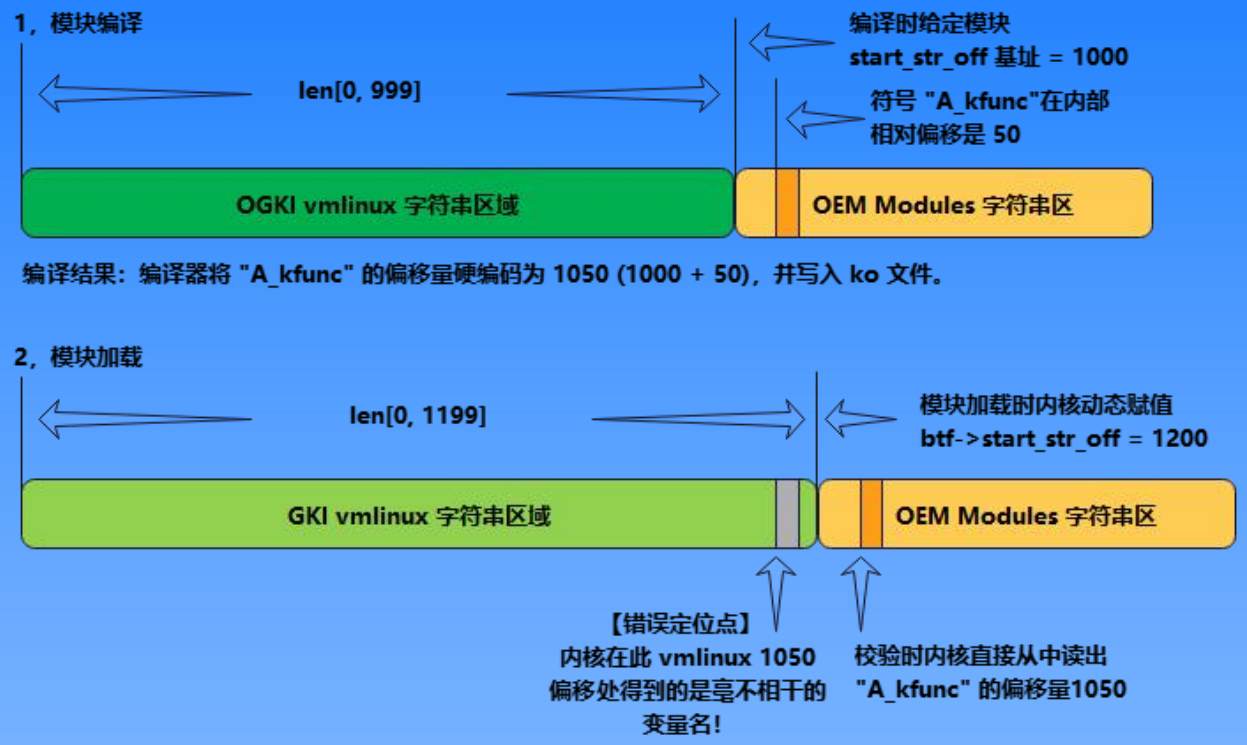
编译用户代码目标文件：

```
clang -c simple.c -o simple.o –lxxx …
```

Android 版本编译构建采用的环境规则下，定制模块xxx.ko（对eBPF scheduler扩展kfunc引入）中符号信息与GKI（Generic Kernel Image）vmlinux表匹配异常， 模块加载时报错：

Kernel module BTF mismatch detected, BTF debug info may be unavailable for some modules.





解决思路：

1，保证ko和vmlinux的编译处于同一代码环境；

2，更新pahole工具绕开ko中符号的偏移量硬编码；

Thanks！